

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

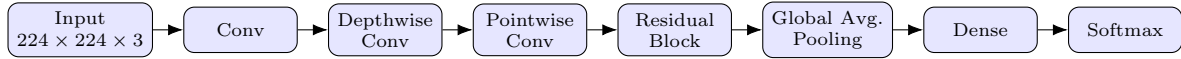
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

The effect of four weight initialization methods was studied:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Analysis

Initialization	Best Validation Accuracy	Final Validation Accuracy
Zero	91.17%	91.17%
Random	90.76%	90.08%
Xavier	90.62%	90.49%
He	90.76%	90.49%

Plot 1: Training Loss vs. Epoch

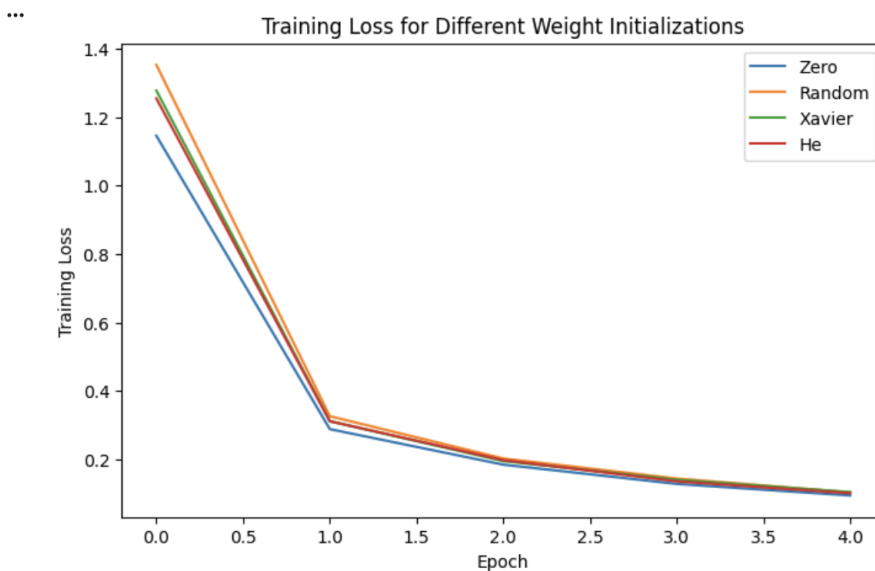


Figure 1: Training Loss vs. Epoch for Different Weight Initializations

Inference: The training loss decreases for all four initialization methods. Zero initialization starts with the lowest loss of 1.1455, while Random initialization starts with the highest loss of 1.3525. Zero also shows faster learning in the beginning as its loss drops to 0.2883 in Epoch 2. By Epoch 5, the loss values become close for all the methods.

Plot 2: Validation Accuracy vs. Epoch

...

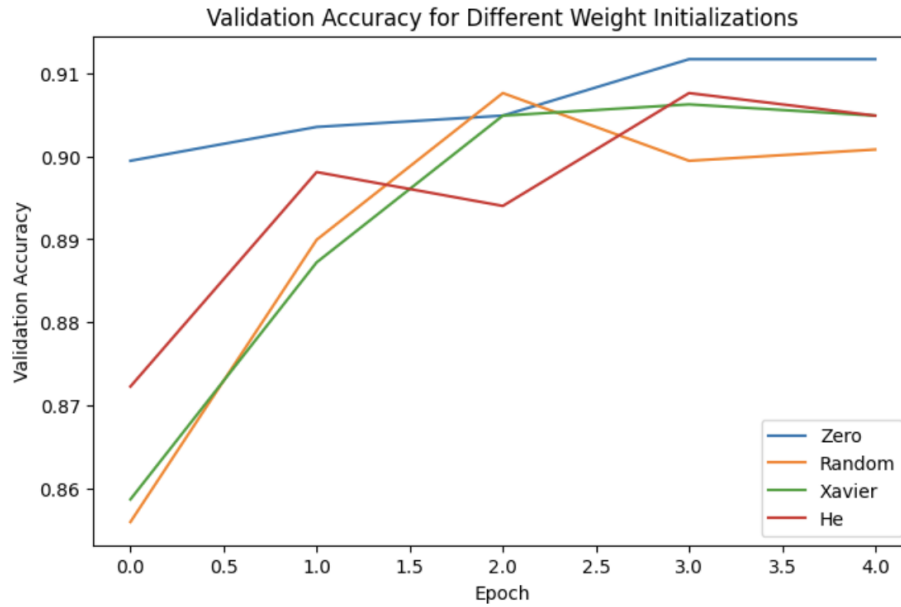


Figure 2: Validation Accuracy vs. Epoch for Different Weight Initializations

Inference: Zero initialization gives the highest validation accuracy of 91.17% and reaches this value by Epoch 4. Random, Xavier and He all reach 90.76%, 90.62% and 90.76% respectively. Zero shows faster and more stable convergence in this experiment because its accuracy increases steadily and remains at 91.17% in the last epoch, while other methods show small fluctuations.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Four regularization approaches were compared:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Plots

Plot 3: Training and Validation Accuracy vs. Epoch

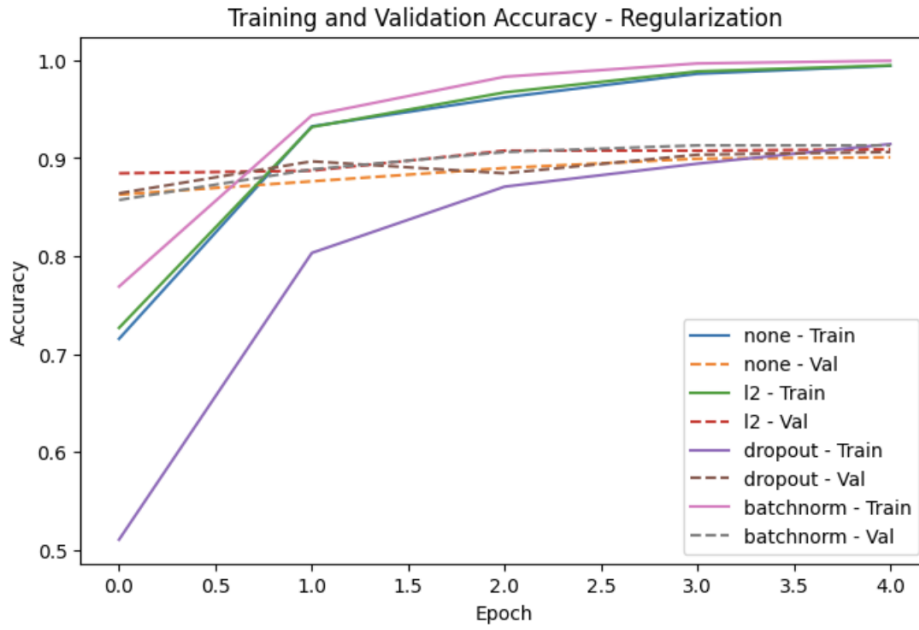


Figure 3: Training and Validation Accuracy for Different Regularization Methods

Inference: The training accuracy increases for all the regularization methods, while the validation accuracy increases more slowly. A gap can be seen between training and validation accuracy, which shows some overfitting. Regularization helps reduce this gap by preventing the model from depending too much on the training data.

Plot 4: Training and Validation Loss vs. Epoch

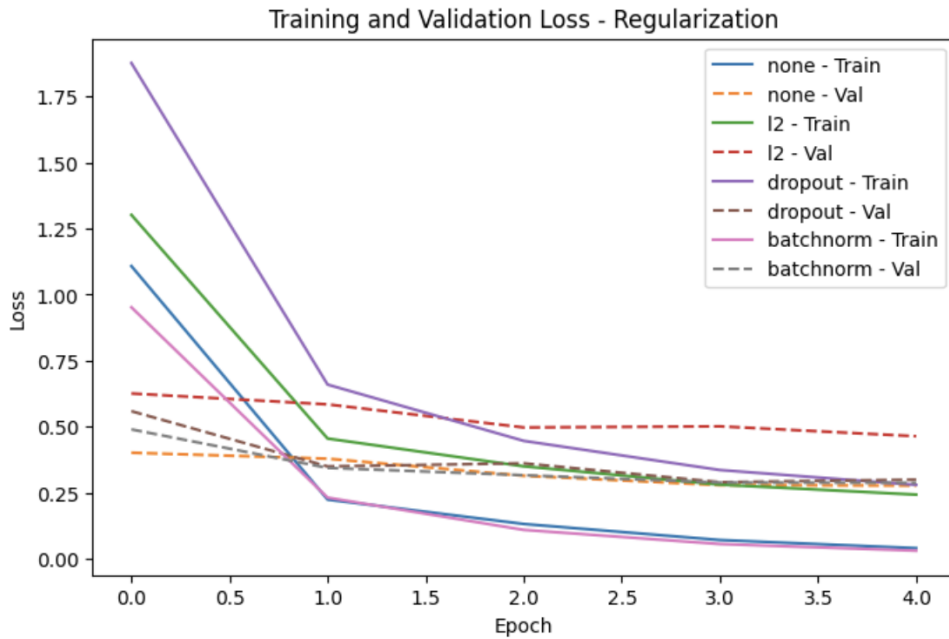


Figure 4: Training and Validation Loss for Different Regularization Methods

Inference: The training loss decreases for all the methods, and the validation loss also generally decreases during the five epochs. Since the validation loss does not show a clear continuous increase while the training loss decreases, strong overfitting is not clearly visible from these curves. The difference between the training and validation losses shows a generalization gap, but it should not be treated as overfitting by itself.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2 - 5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4 - 5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6 - 5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8 - 5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

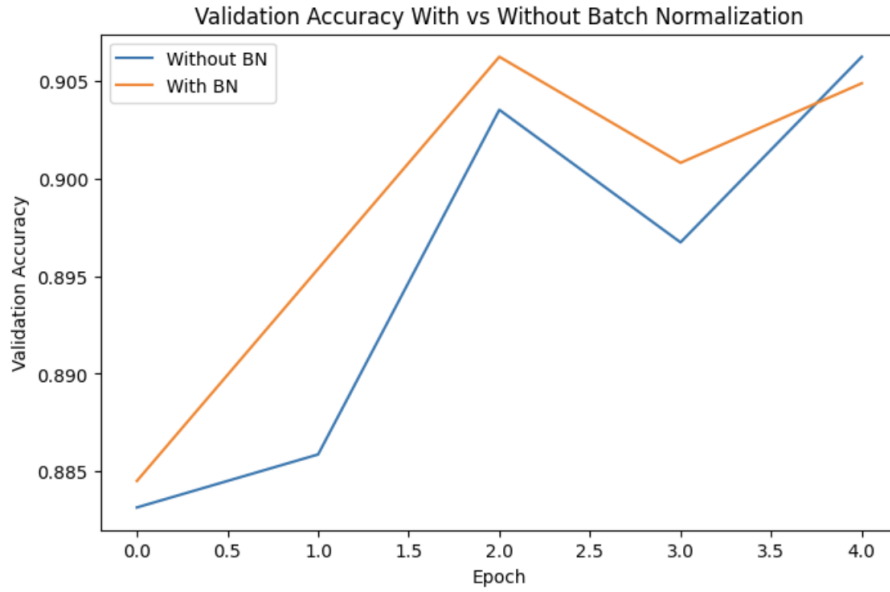


Figure 5: Validation Accuracy With and Without Batch Normalization

Inference: The validation accuracy with Batch Normalization is slightly better than the model without Batch Normalization in our experiment. This shows that BN helped the model learn more stable features during training. The normalization of activations makes the training process more stable and can improve generalization.

8. Optimization Algorithms

Four optimization algorithms were compared: SGD, Momentum, RMSProp and Adam. All four optimizers used a learning rate of 0.001 and were trained for five epochs. The objective was to compare their convergence speed and validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

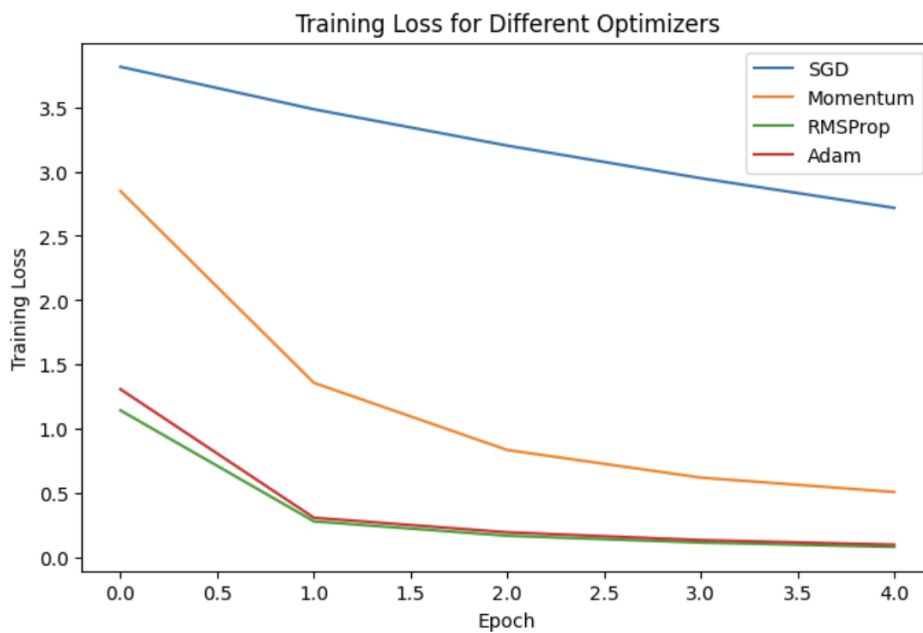


Figure 6: Training Loss for Different Optimizers

Inference: The training loss decreases for all four optimizers, but the rate of the decrease is different. SGD decreases the loss slowly and still has a relatively high loss after five epochs, while Momentum, RMSProp and Adam converge much faster. RMSProp reaches the lowest final training loss of 0.0843.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

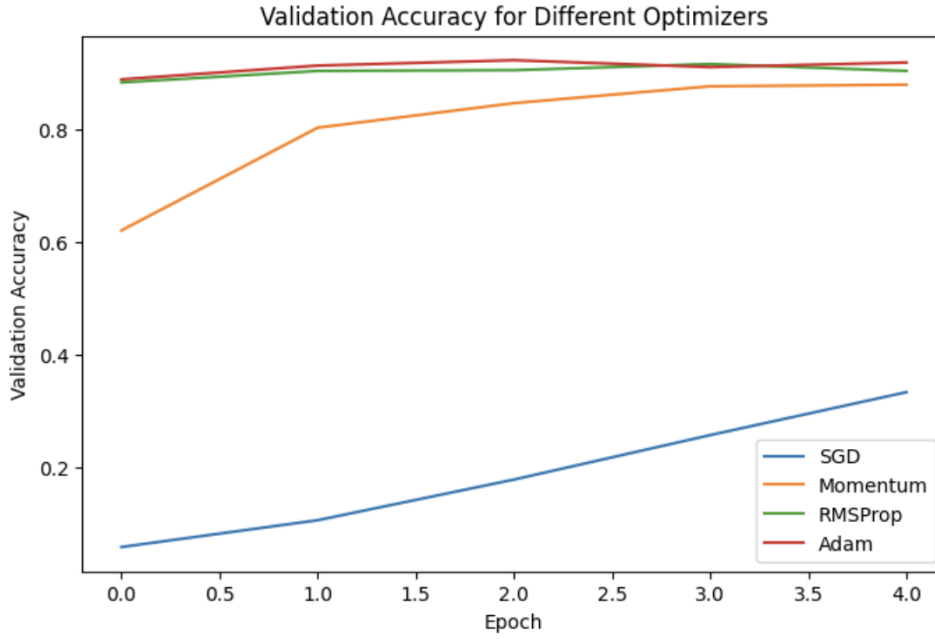


Figure 7: Validation Accuracy for Different Optimizers

Inference: Adam reaches the highest validation accuracy of 92.26%, followed by RMSProp at 91.58%. Momentum reaches 87.91%, while SGD reaches only 33.42% after five epochs. Adam reaches its best validation accuracy by the third epoch, showing a faster convergence in this experiment.

Optimizer	Final Loss	Best Val. Accuracy
SGD	2.7177	33.42%
Momentum	0.5107	87.91%
RMSProp	0.0843	91.58%
Adam	0.1013	92.26%

Result: Adam gives the highest validation accuracy of 92.26% and reaches its best validation accuracy by epoch 3. Therefore, Adam was selected as the optimizer for the subsequent hyperparameter experiments.

9. CNN Hyperparameter Tuning

The effect of learning rate, batch size and dropout rate was studied while keeping the other settings fixed. Adam was used as the optimizer for these experiments. Each experiment was conducted for five epochs.

Hyperparameter	Values Tested
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	Adam
Fine-Tuning Learning Rate	10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

Learning Rate

Plot 8: Learning Rate vs. Validation Accuracy

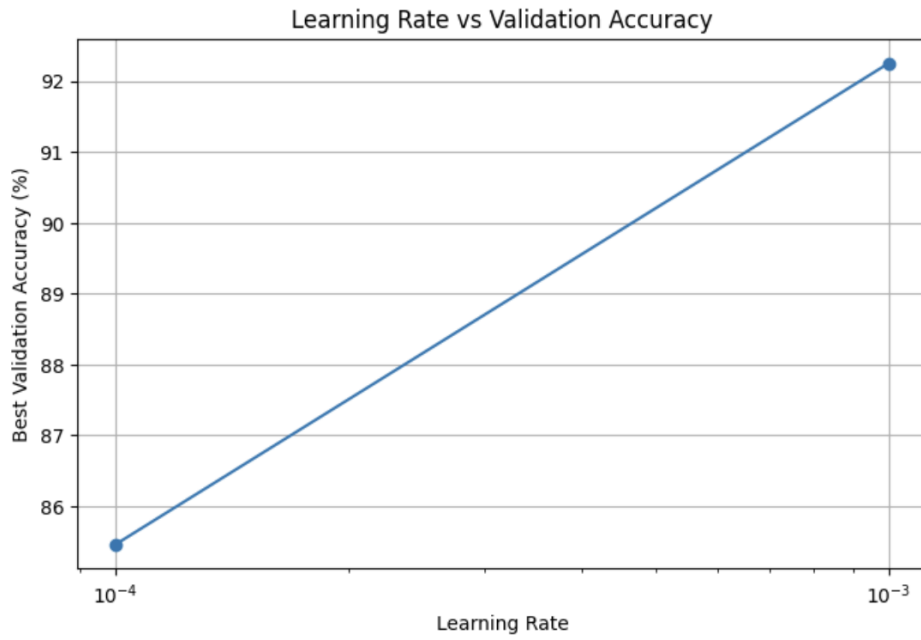


Figure 8: Learning Rate vs. Validation Accuracy

Learning Rate	Best Val. Accuracy	Best Epoch	Final Train Loss	Final Val. Loss
0.001	92.26%	5	0.1048	0.2775
0.0001	85.46%	5	0.9134	0.8885

Inference: The learning rate of 0.001 performs substantially better than 0.0001, achieving 92.26% validation accuracy compared with 85.46%. The smaller learning rate produces much smaller parameter updates, so the model learns more slowly within the five available epochs.

Batch Size

Plot 9: Batch Size vs. Validation Accuracy

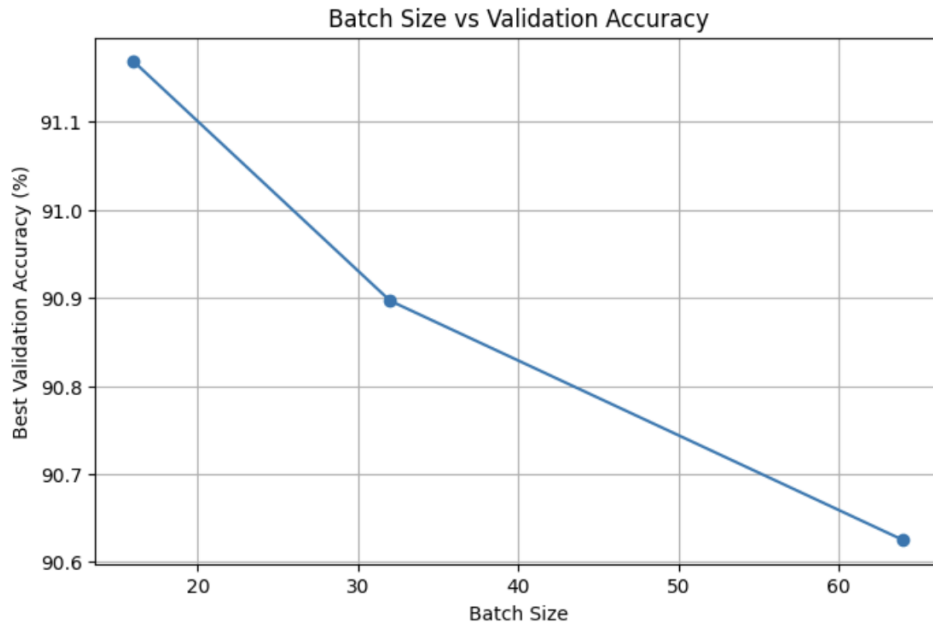


Figure 9: Batch Size vs. Validation Accuracy

Batch Size	Best Val. Accuracy	Best Epoch	Final Train Loss	Final Val. Loss
16	91.17%	5	0.0652	0.2619
32	90.90%	5	0.1009	0.2833
64	90.63%	5	0.1582	0.3217

Inference: The best validation accuracy is obtained with a batch size of 16. Validation accuracy decreases slightly as the batch size increases from 16 to 64. The smaller batch size provides more frequent parameter updates during training.

Dropout Rate

Plot 10: Dropout Rate vs. Validation Accuracy

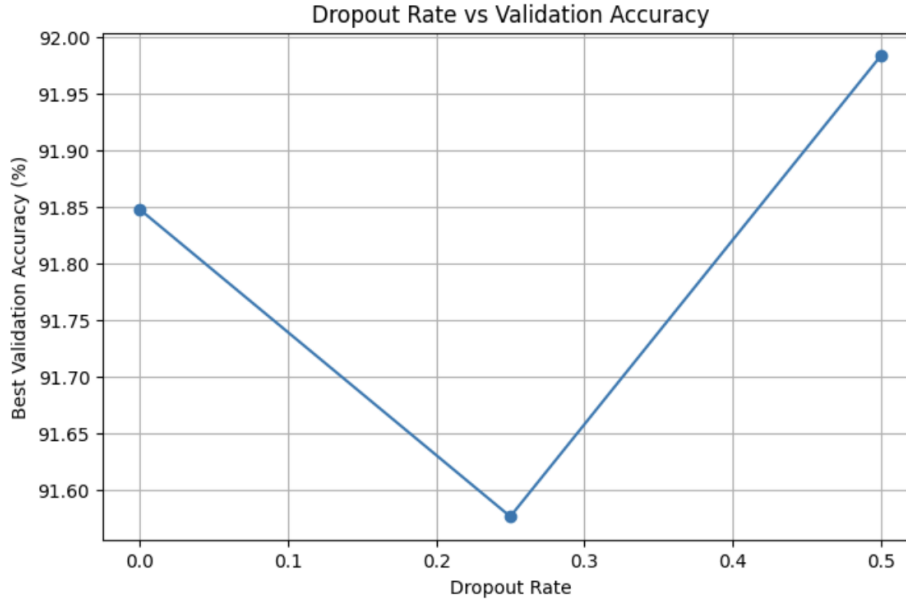


Figure 10: Dropout Rate vs. Validation Accuracy

Dropout Rate	Best Val. Accuracy	Best Epoch	Final Train Loss	Final Val. Loss
0	91.85%	5	0.1002	0.2750
0.25	91.58%	5	0.1467	0.2779
0.5	91.98%	3	0.2246	0.2914

Inference: The three dropout settings give similar validation accuracies. The dropout rate of 0.5 gives the highest validation accuracy of 91.98%, and it reaches this value by epoch 3. Therefore, 0.5 is selected as the best dropout rate from this experiment.

Selected Hyperparameters

Hyperparameter	Selected Value
Optimizer	Adam
Learning Rate	0.001
Batch Size	16
Dropout Rate	0.5
Fine-Tuning Learning Rate	10^{-5}

Result: Based on the individual experiments, Adam with a learning rate of 0.001, batch size of 16 and dropout rate of 0.5 provides a strong combination of hyperparameters. These values were subsequently included in the candidate configurations used for cross-validation.

10. Transfer Learning and Fine-Tuning

Transfer learning was performed using MobileNetV2 pretrained on ImageNet. First, feature extraction was performed by freezing all 154 MobileNetV2 layers and training the newly added classifier. The model was then fine-tuned by unfreezing the last 30 layers and using a smaller learning rate of 10^{-5} .

Case A: Feature Extraction

In feature extraction, all MobileNetV2 layers were frozen and only the classification layers were trained. The model was trained for five epochs using Adam with a learning rate of 0.001. At the end of Epoch 5, the training accuracy was 94.23%, validation accuracy was 90.35%, training loss was 0.2184 and validation loss was 0.2952.

Case B: Fine-Tuning

For fine-tuning, the last 30 of the 154 MobileNetV2 layers were unfrozen. The learning rate was reduced to 10^{-5} so that the pretrained features could be adjusted with smaller updates. At the end of Epoch 5, the training accuracy was 93.07%, validation accuracy was 90.76%, training loss was 0.2290 and validation loss was 0.2701.

Method	Trainable MobileNetV2 Layers	Final Train. Accuracy	Final Val. Accuracy	Final Val. Loss
Feature Extraction	0	94.23%	90.35%	0.2952
Fine-Tuning	30	93.07%	90.76%	0.2701

Plot 11: Feature Extraction vs. Fine-Tuning

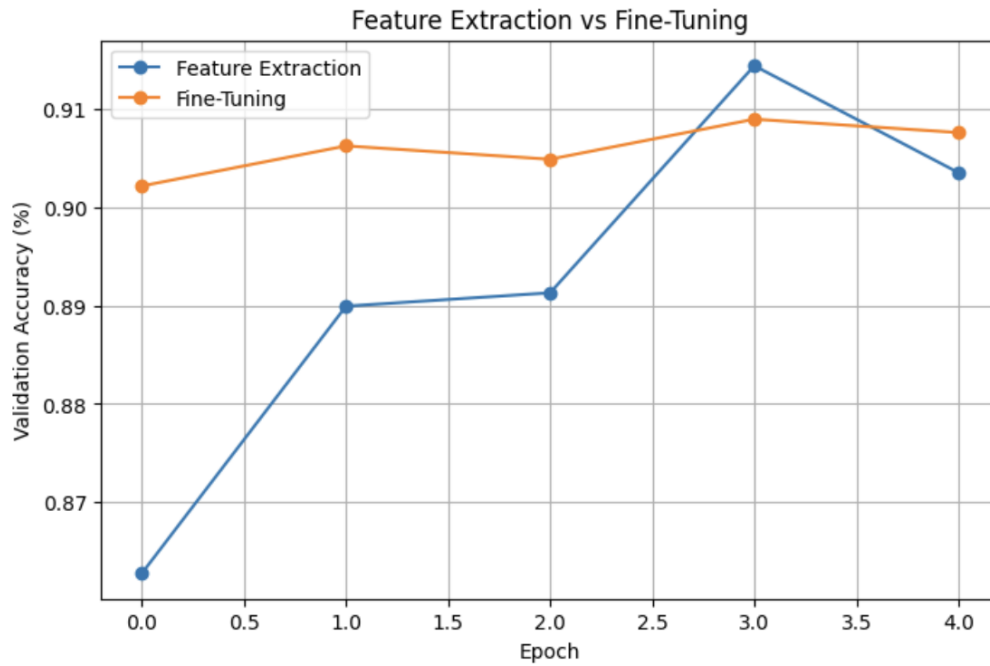


Figure 11: Feature Extraction vs. Fine-Tuning

Inference: At the end of five epochs, fine-tuning gives a slightly higher validation accuracy of 90.76% compared with 90.35% for feature extraction. This is only a small improvement of 0.41 percentage points, but it shows that unfreezing the last 30 layers allowed the pretrained features to adapt slightly better to the pet dataset. The training accuracy is lower for fine-tuning, which is also expected because the pretrained layers are being updated instead of keeping all the learned features fixed.

Plot 12: Training and Validation Loss Before and After Fine-Tuning

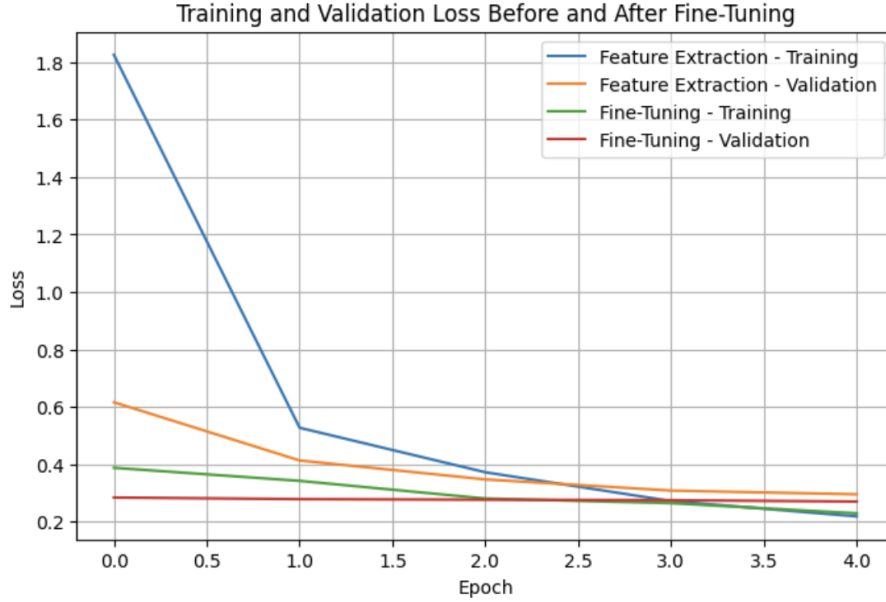


Figure 12: Training and Validation Loss for Feature Extraction and Fine-Tuning

Inference: At Epoch 5, fine-tuning gives a lower validation loss of 0.2701 compared with 0.2952 for feature extraction. This means that the fine-tuned model is slightly better at the end of training even though its training accuracy is lower. The smaller learning rate of 10^{-5} allows the pretrained layers to make small changes, which helps adapt the already learned features to the new dataset without changing them too aggressively.

Result: Based on the final Epoch 5 results, fine-tuning performs slightly better than feature extraction, with 90.76% validation accuracy compared with 90.35%. The improvement is small, but the lower validation loss of 0.2701 also supports the fine-tuning result. Therefore, fine-tuning the last 30 layers gave slightly better final performance in this experiment.

11. K-Fold Cross-Validation

After the individual experiments, four promising configurations were selected for 5-fold stratified cross-validation. The cross-validation was performed on 2944 training images. The independent test set was not used during hyperparameter selection.

Configuration	Optimizer	Learning Rate	Batch Size	Dropout
C1	Adam	0.001	32	0
C2	Adam	0.001	16	0
C3	Adam	0.001	32	0.5
C4	Adam	0.001	16	0.5

Each configuration was evaluated on five validation folds.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	89.98%	90.49%	90.15%	90.32%	90.82%	90.35% $\pm$ 0.29%
C2	90.49%	91.00%	90.15%	90.15%	90.14%	90.39% $\pm$ 0.34%
C3	90.15%	90.32%	90.83%	91.17%	92.86%	91.07% $\pm$ 0.97%
C4	90.49%	91.17%	89.13%	89.13%	91.16%	90.22% $\pm$ 0.92%

Plot 13: 5-Fold Cross-Validation Accuracy

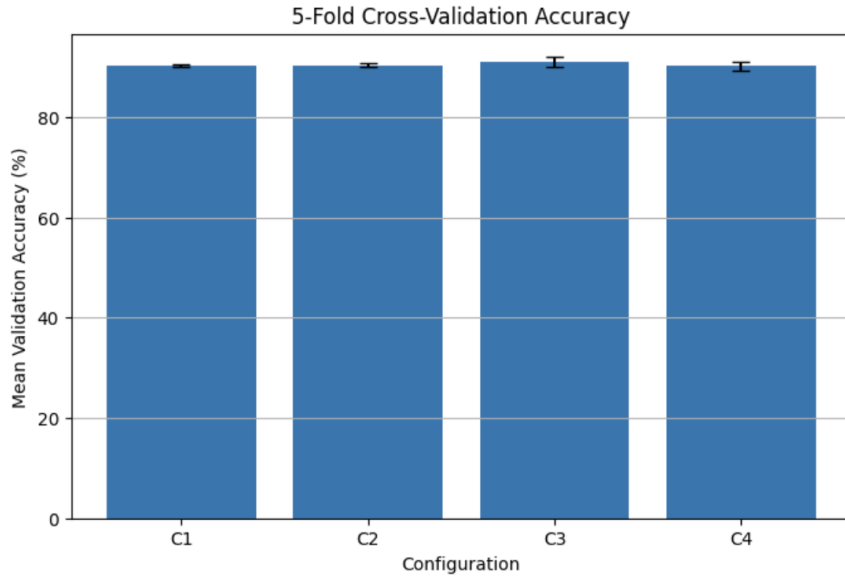


Figure 13: 5-Fold Cross-Validation Accuracy

Inference: C3 obtains the highest mean validation accuracy of 91.07%. Its standard deviation is 0.97%, indicating somewhat greater variation between folds than C1 and C2. However, its improvement in mean accuracy is sufficient for C3 to be selected as the best configuration based on the implemented selection criterion.

Result: Configuration C3 was selected as the best configuration because it achieved the highest mean 5-fold cross-validation accuracy of 91.07%. Its parameters are Adam, learning rate 0.001, batch size 32 and dropout rate 0.5.

12. Final Model Evaluation

After selecting C3 using 5-fold cross-validation, the final model was trained using the complete cross-validation training data. The selected configuration uses Adam with a learning rate of 0.001, batch size 32 and dropout rate 0.5. The final model was trained for 10 epochs and then evaluated on the independent test set.

The final model contains 2,305,381 total parameters, of which 47,397 are trainable and 2,257,984 are non-trainable.

Metric	Value
Mean CV Accuracy	91.07%
CV Standard Deviation	0.97%
Test Accuracy	90.24%
Test Loss	0.3127
Macro Precision	0.9035
Macro Recall	0.9016
Macro F1-score	0.9011
Training Time	74.56 s
Number of Parameters	2,305,381

Plot: Final Model Training and Validation Accuracy

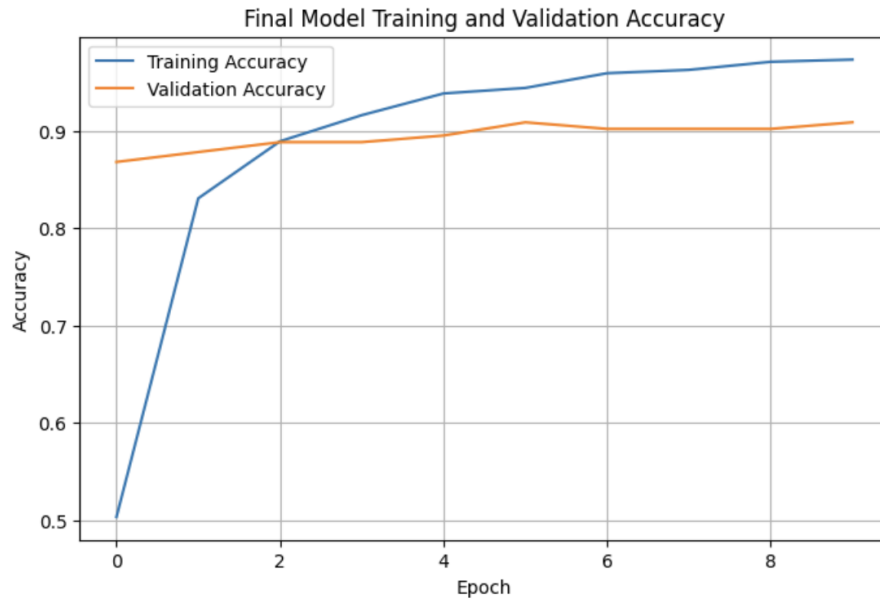


Figure 14: Final Model Training and Validation Accuracy

Inference: The training accuracy increases as the model learns the training images, while validation accuracy shows the model's performance on unseen validation data. The difference between the curves represents the generalization gap. The validation performance remains reasonably high, indicating that the model generalizes well despite some difference from training performance.

Plot: Final Model Training and Validation Loss

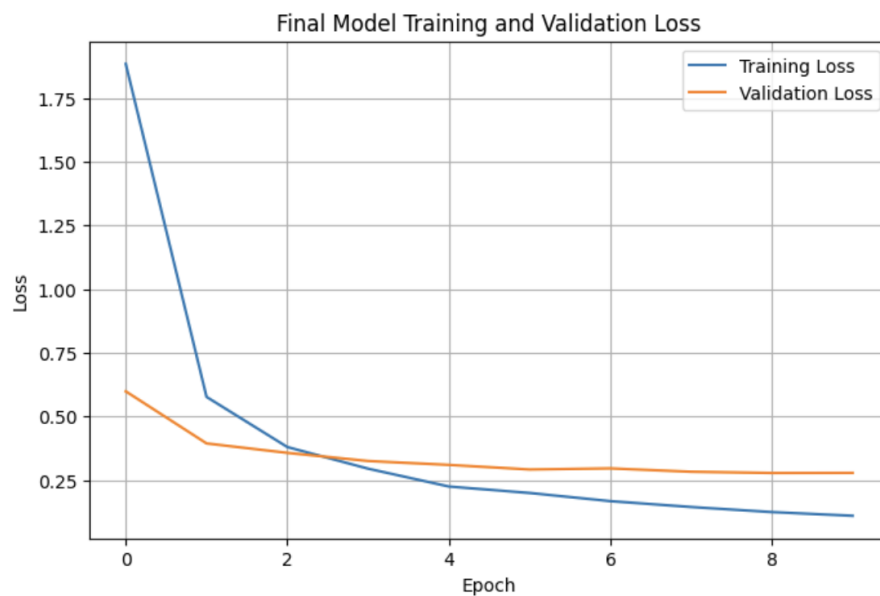


Figure 15: Final Model Training and Validation Loss

Inference: The training loss decreases as the model learns the training data. The validation loss also decreases, showing that the model continues to improve on unseen data during training. The validation loss remains higher than the training loss, which indicates a moderate generalization gap.

Plot 14: Confusion Matrix

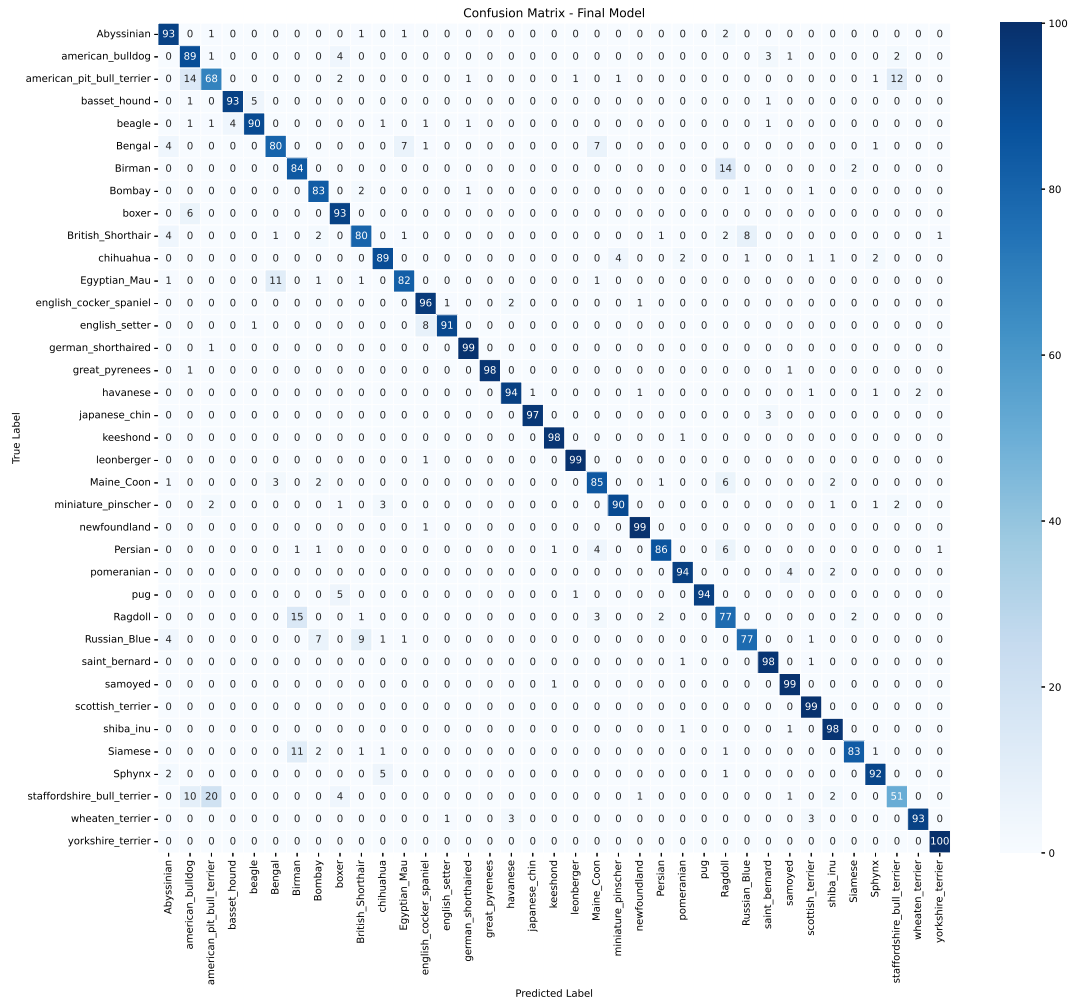


Figure 16: Confusion Matrix of the Final Model

Inference: The confusion matrix shows that most of the values are on the diagonal, which means that most classes are classified correctly. The best classified classes include Yorkshire Terrier with 100 correct predictions, German Shorthaired, Leonberger, Newfoundland, Samoyed and Scottish Terrier with 99 correct predictions, and Great Pyrenees, Keeshond, Shiba Inu and Saint Bernard with 98 correct predictions.

The most frequent confusion is between Staffordshire Bull Terrier and American Pit Bull Terrier. Out of the Staffordshire Bull Terrier images, 20 were predicted as American Pit Bull Terrier, while 12 American Pit Bull Terrier images were predicted as Staffordshire Bull Terrier. Other noticeable confusions are Ragdoll being predicted as Birman for 15 images and Birman being predicted as Ragdoll for 14 images. These errors can happen because some breeds have similar body shape, face structure, fur pattern, colour and overall appearance, making them harder to distinguish from the images alone.

Overall, the strong diagonal and the relatively smaller off-diagonal values show that the final model performs well for most classes, but visually similar breeds are still more difficult for the model to separate.

Final Result: The final model achieves a test accuracy of 90.24%, with macro precision of 0.9035, macro recall of 0.9016 and macro F1-score of 0.9011. The results show that the selected MobileNetV2 configuration provides strong performance on the independent test set.

13. Overall Results

The different experiments were compared using their best validation accuracy and, where recorded, training time. The final selected configuration was further evaluated using 5-fold cross-validation and an independent test set.

The different experiments were compared based on their best validation accuracy. The main results are shown below.

Configuration	Best Validation Accuracy
Baseline	90%
Best Initialization (Zero)	91.17%
Best Regularization (Dropout 0.5)	91.98%
Best Optimizer (Adam)	92.26%
Best Hyperparameters (C3)	91.07%
Feature Extraction	91.44%
Fine-Tuned Model	90.90%

Overall Observation:

The experiments show that the choice of training strategy had a noticeable effect on model performance. Zero initialization gave the highest validation accuracy among the tested initialization methods, achieving 91.17%. Among the optimizers, Adam performed best with a validation accuracy of 92.26%.

For regularization, a dropout rate of 0.5 gave the highest validation accuracy of 91.98% among the tested dropout settings. The hyperparameter experiments also showed that Adam with a learning rate of 0.001, batch size 16 and dropout rate 0.5 provided a strong combination of settings. The dropout experiment results show 91.98% validation accuracy for a dropout rate of 0.5.

Four promising configurations were subsequently evaluated using 5-fold cross-validation. C3 achieved the highest mean accuracy of 91.07% with a standard deviation of 0.97%. The selected configuration was then retrained and evaluated on the independent test set, achieving 90.24% accuracy. The final model also achieved macro precision of 0.9035, macro recall of 0.9016 and macro F1-score of 0.9011.

For transfer learning, the final epoch results show that fine-tuning performed slightly better than feature extraction. Feature extraction achieved a final validation accuracy of 90.35%, while fine-tuning the last 30 layers achieved 90.76%. Fine-tuning also had a lower final validation loss of 0.2701 compared with 0.2952 for feature extraction. This shows that allowing some of the pretrained layers to adapt to the pet dataset gave a small improvement in the final epoch.

Overall, C3 was selected as the final model because it provided the best mean cross-validation accuracy among the candidate configurations while maintaining relatively low training time. Its 90.24% independent test accuracy also demonstrates good generalization to unseen data.

14. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters are the values which the model learns by itself during training, such as weights and biases. Hyperparameters are values which we choose before training, such as learning rate, batch size, dropout rate and number of epochs. For example, in our experiment the weights of MobileNetV2 were learned during training, while we selected the learning rate as 0.001 and batch size as 16 or 32.

2. Why is weight initialization important?

Weights need some starting values before the training can begin. A suitable initialization helps the model start learning properly and can make the training more stable. We observed this in the initialization experiment, where the different starting values produced different learning curves and validation accuracies during the five epochs.

3. Why can zero initialization be problematic for neural networks?

If all neurons in the same layer start with zero weights, they can receive the same updates and learn the same features. This breaks the idea of having different neurons learn different patterns. However, in our experiment zero initialization gave the highest validation accuracy of 91.17% because we were mainly initializing the newly added classifier while the pretrained MobileNetV2 layers were frozen. So the usual problem of zero initialization was not strongly visible in this particular experiment.

4. Compare Xavier and He initialization.

Xavier initialization is designed to keep the variance of activations reasonably balanced between layers and is commonly used with sigmoid or tanh. He initialization uses a different scaling and is more suitable for ReLU based networks. In our experiment, Xavier and He gave very close validation results, so there was no large difference between them for this model setup.

5. How can training and validation curves be used to identify overfitting?

The training curve shows how well the model is learning the training data, while the validation curve shows how it performs on unseen validation data. If training accuracy keeps increasing but validation accuracy stops improving or decreases, the model may be overfitting. Similarly, if training loss keeps decreasing while validation loss starts increasing, the model is learning the training data too specifically.

6. How does Dropout reduce overfitting?

Dropout randomly switches off some neurons during training. Because the model cannot depend on the same neurons all the time, it is forced to learn more general patterns. This can reduce the gap between training and

validation performance. In our experiment, different dropout rates gave slightly different validation accuracies, showing that the amount of dropout also has to be selected properly.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes the activations of a layer using the values in the current mini-batch. This keeps the values in a more controlled range and can make the training process more stable. In our experiment, adding Batch Normalization changed the training behaviour, although it did not give a clear accuracy improvement in the separate BN comparison.

8. Explain the numerical Batch Normalization example.

For the values $[2, 4, 6, 8]$, the mean is 5 and the variance is 5. The standard deviation is therefore $\sqrt{5} \approx 2.236$. Subtracting the mean and dividing by the standard deviation changes the values to approximately $[-1.342, -0.447, 0.447, 1.342]$. This shows how Batch Normalization changes the values so that they are centered around zero and have a controlled scale.

9. What are the roles of γ and β ?

After normalization, γ is used to scale the normalized value and β is used to shift it. They are learnable parameters, so the network can decide how much scaling and shifting is useful. For example, even though Batch Normalization first centers the values, the network can learn a different useful range using γ and β .

10. Compare SGD, Momentum, RMSProp and Adam.

SGD directly uses the current gradient to update the weights, so its updates can be slower. Momentum also uses information from previous updates, which helps the model move more smoothly. RMSProp changes the learning rate based on recent gradient values, while Adam combines adaptive learning rates with momentum. We observed a clear difference in our experiment: SGD reached only 33.42% validation accuracy, while RMSProp reached 91.58% and Adam reached 92.26%, so Adam was selected for the later experiments.

11. What happens when the learning rate is too large?

A large learning rate makes the model change its weights by large amounts at each update. Because of this, the model can jump over a good solution instead of slowly moving towards it. The loss can become unstable or the model may fail to converge properly. Therefore, the learning rate has to be large enough to learn but not so large that the updates become uncontrolled.

12. What happens when the learning rate is too small?

A very small learning rate makes every weight update very small. Because of this, the model learns very slowly and may need many more epochs to reach a good solution. We saw this in our experiment: 0.0001 gave only 85.46% validation accuracy, while 0.001 reached 92.26% in the same five epochs.

13. What is the effect of increasing batch size?

Increasing the batch size means more training samples are used before one weight update is made. This usually makes the gradient estimate smoother, but it also means fewer updates are made during each epoch. In our experiment, batch size 16 gave 91.17% validation accuracy, while 32 and 64 gave 90.90% and 90.63%. So increasing the batch size did not help our model in this experiment.

14. Explain stride and padding.

Stride tells how far the convolution filter moves each time over the image. For example, stride 2 moves the filter two pixels at a time and produces a smaller output. Padding adds extra pixels around the image before convolution. It is useful because it allows the filter to also process the border pixels and can help maintain the required output size.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 is designed as a lightweight CNN, so it uses much less computation than many large CNN architectures. One important reason is its use of depthwise separable convolution. This makes it suitable for applications where memory and computation are limited while still providing useful image features.

16. What is depthwise separable convolution?

A normal convolution performs filtering and channel combination together. Depthwise separable convolution separates these operations into two steps. First, a separate filter is applied to each input channel and then a 1×1 convolution combines the channels. This greatly reduces the number of calculations compared with a normal convolution.

17. What is transfer learning?

Transfer learning means starting with a model which has already learned useful features from a large dataset and using those features for a new task. In our experiment, MobileNetV2 pretrained on ImageNet was used for the Oxford-IIIT Pet classification task. This avoids having to learn all the image features from the beginning.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained MobileNetV2 layers are frozen and only the newly added classification part is trained. In fine-tuning, some of the pretrained layers are unfrozen and trained along with the classifier. In our experiment, feature extraction gave 91.44% validation accuracy, while fine-tuning the last 30 layers gave 90.90%. So allowing the pretrained features to change did not improve the result in our experiment.

19. Why is a smaller learning rate generally used during fine-tuning?

The pretrained network already contains useful features learned from ImageNet. If a large learning rate is used, the weights can change too much and some of these useful features may be lost. Therefore, a smaller learning

rate is used to make small adjustments instead of changing the pretrained features completely. We used 10^{-5} for fine-tuning for this reason.

20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**

With only one train-validation split, the result can depend too much on which images happened to be placed in the validation set. K-Fold Cross-Validation uses different validation folds, so the same configuration is tested several times. In our experiment, the four configurations were compared over five folds, giving a mean accuracy and standard deviation for each configuration.

21. **Why must the test set remain untouched during tuning?**

The test set is supposed to represent completely unseen data. If we use the test results while changing the hyperparameters, we indirectly use information from the test set to make our decisions. This can make the final test accuracy look better than the actual generalization performance. Therefore, we used the validation data and cross-validation for selection and kept the independent test set for the final evaluation.

22. **Why should mean and standard deviation both be reported?**

The mean tells us the average accuracy obtained across all the folds. However, the mean alone does not tell us whether the model gives similar results on every fold. Standard deviation tells us how much the accuracy changes from one fold to another. For example, our selected C3 configuration had a mean accuracy of 91.07% with a standard deviation of 0.97%, so we know both its average performance and its variation across the folds.

23. **Is the highest validation accuracy always sufficient to select a model?**

No. A model can get a high validation accuracy but still have high variation, overfit the training data or perform poorly on completely unseen data. We therefore also looked at the cross-validation mean and standard deviation before selecting the final configuration. In our experiment, C3 was selected because it had the highest mean 5-fold accuracy of 91.07%, and its final test accuracy was 90.24%.

15. Source Code

The complete source code and experiment notebook are available in the following GitHub repository:

https://github.com/srinithi0406/Deep_Learning/tree/main/DL_5

16. Conclusion

In this experiment, the effect of different CNN training strategies was studied using MobileNetV2 and the Oxford-IIIT Pet dataset. Weight initialization, regularization, Batch Normalization, optimization algorithms, hyperparameter tuning, transfer learning, fine-tuning and cross-validation were investigated.

The optimizer comparison showed that Adam performed the best, achieving a validation accuracy of 92.26% after five epochs. Among the regularization methods, Batch Normalization achieved the highest validation accuracy of 91.30%. The learning-rate experiment showed that 0.001 performed substantially better than 0.0001, while the batch-size and dropout experiments showed that smaller batch sizes and suitable dropout values can provide better validation performance. Based on the final epoch results, fine-tuning performed slightly better than feature extraction, with 90.76% validation accuracy compared with 90.35%. The improvement is small, but the lower validation loss also supports the fine-tuning result in this experiment. Finally, four candidate configurations were compared using 5-fold cross-validation. Configuration C3, consisting of Adam, learning rate 0.001, batch size 32 and dropout 0.5, achieved the highest mean cross-validation accuracy of 91.07% with a standard deviation of 0.97%. The final model trained using this configuration achieved 90.24% accuracy on the independent test set, with macro precision of 0.9035, macro recall of 0.9016 and macro F1-score of 0.9011.

Thus, the experiment demonstrates that careful selection of optimization, regularization and hyperparameters, together with cross-validation, is important for obtaining a reliable deep learning model.

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>